

CUED - Engineering Tripos Part IIA 2023-2024

Module Laboratory Report

Student name:	Yongqing Jiang	CRSID:	yj375	Module number:	3B2
---------------	----------------	--------	-------	----------------	-----

Feedback to the student

☐ See also comments in the text

		Very good	Good	Needs improvmt
C O N T E N T	Completeness, quantity of content: Has the report covered all aspects of the lab? Has the analysis been carried out thoroughly?			
	Correctness, quality of content Is the data correct? Is the analysis of the data correct? Are the conclusions correct?			
	Depth of understanding, quality of discussion Does the report show a good technical understanding? Have all the relevant conclusions been drawn?			
	Comments:			
P R E S E N T A T I O N	Attention to detail, typesetting and typographical errors Is the report free of typographical errors? Are the figures/tables/references presented professionally?			
	Comments:			

Raw report mark	/ 5
Penalty for lateness	

The weighting of comments is not intended to be equal, and the relative importance of criteria may vary between modules. A good report should attract 4 marks.

*1 mark / week or part week.
Please refer to the [online](#) information regarding our extension policy.*

Marker:

Date:

CAMBRIDGE UNIVERSITY ENGINEERING

DEPARTMENT

Part IIA Laboratory Report

3B2 FPGA Programming and Testing

Name: Yongqing Jiang

CRSid: yj375

College: Peterhouse

Date of Experiment: Feb. 2026

1 Introduction

This lab activity attempts to implement a logic function that was shown on the lecture on a DE1-SoC development board, which presents a hardware design platform built on Altera Cyclone V FPGA. The provided verilog code is compiled and uploaded to the development board via Quartus Prime, in which pin assessment is available. The FPGA board was tested under different user input. Lastly, some improvements are made to adapt to real-world situations.

2 Problem Description and Design

This design aims to model a controller of pedestrian light-controlled crossing, as shown by Figure 1.

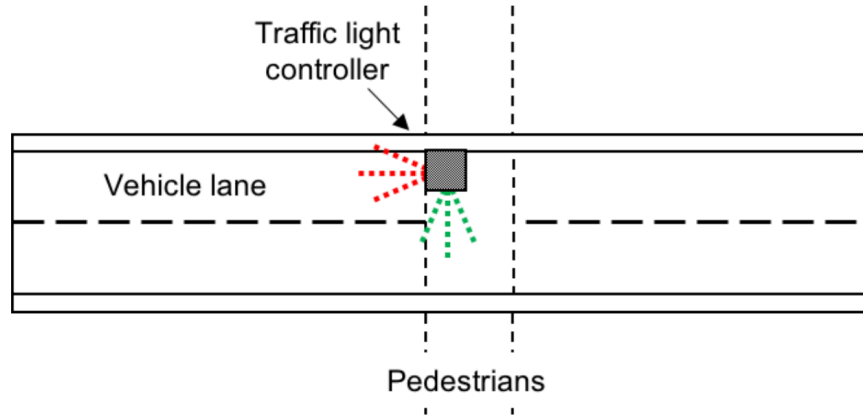


Figure 1: Pedestrian light-controlled crossing

In this scenario, the traffic light controller controls 2 lights: pedestrian light and vehicle light. The vehicle light has three valid states: RED, YELLOW, and GREEN, while the pedestrian light has two valid states: RED, and GREEN.

The external inputs include REQUEST from pedestrian-pressed button, RESET to ensure the robustness of the system, and two timers T5 and T10 to enable internal time recording.

The requirement of the traffic light control system is given as:

1. The initial state is vehicle GREEN and pedestrian RED;
2. It can only change state by demand, i.e. a pedestrian request or reset;
3. After a request is made, the following 3-state sequence needs to be implemented:
 - It starts by turning vehicle to YELLOW for 5 seconds;
 - Then it turns vehicle to RED and pedestrian to GREEN at the same time, both for 10 seconds;
 - Then it returns to the initial state.
4. A reset command returns the system to its initial state.

The state transition diagram is shown by Figure 2. It is important to note that the problem is reduced to have only 3 states labelled by the current vehicle light. This is because the pedestrian light is dependent on vehicle light.

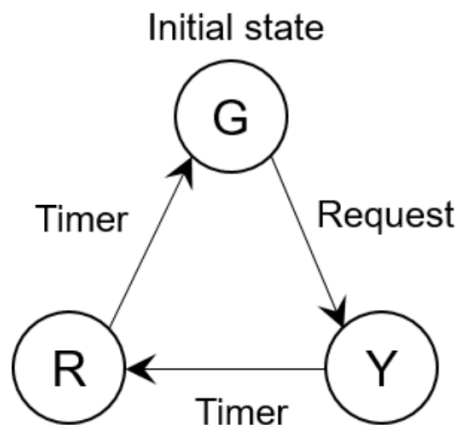


Figure 2: State diagram of the problem

From the state diagram, a state table can be constructed as shown in Table 1.

State	Next state				Vehicle lights			Pedestrian lights	
	Reset	Request	5 s	10 s	GREEN	YELLOW	RED	GREEN	RED
G	G	Y			1	0	0	0	1
Y	G		R		0	1	0	0	1
R	G			G	0	0	1	1	0

Table 1: Traffic Light State Transition Table

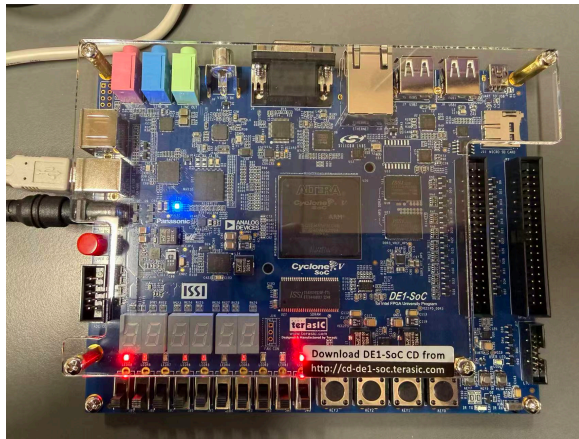
3 FPGA Implementation and Improvement

The verilog code which could implement this design is provided by the lab handout, which is included in the Appendix by Listing 2.

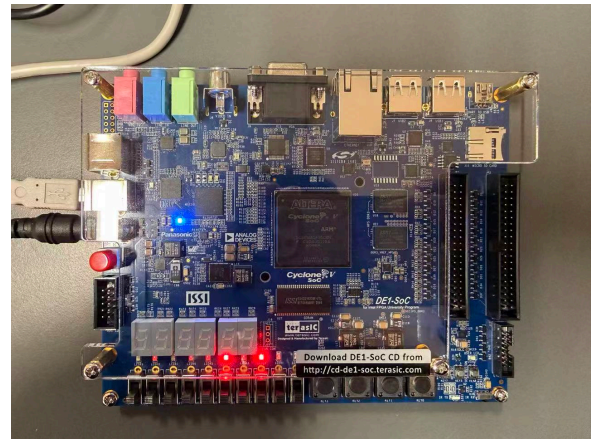
To implement this verilog code on hardware platform, Quartus Prime is needed to compile and upload code to DE1-SoC.

The key steps in Quartus Prime include compiling verilog code, pin assignment, configuring FPGA device, and uploading the code to FPGA.

After that, the designed circuit is tested by pressing buttons that are assigned to give REQUEST and RESET signal, and observing the LED output that corresponds to 3 vehicle lights and 2 pedestrian lights.



(a) State G: initial state



(b) State R: state 5s after REQUEST

Figure 3: Picture of FPGA board under operation

Shown by Figure 3, 5 of 10 available LEDs are assigned to 5 lights. The correspondence is:

1. LED0: Pedestrian RED
2. LED2: Pedestrian GREEN
3. LED4: Vehicle RED
4. LED7: Vehicle YELLOW
5. LED9: Vehicle GREEN

These lights are set to have space to each other to simplify observation.

By pressing REQUEST and RESET buttons on different state, the observation shows that the FPGA demonstrates the required design objective.

However, this design has a drawback. If the pedestrian presses the button for multiple times, the Finite State Machine would always initiate and it would never stays at State G. Which means that the vehicle light never turns green, blocking the traffic.

This could be improved by slight modification on the condition of transition from State G to Y. The original code from line 18 to 23 is modified as shown by Listing 1.

```
1  reg req_latch;
2  G: begin
3      if (req == 1'b0) begin
4          req_latch <= 1'b1;
5      end
6      if (count < 29'd500000000 ) begin
7          count <= count + 29'd1;
8      end
9      if ((count >= 29'd500000000) && ((req == 1'b0) || (
10         req_latch == 1'b1))) begin
11         state <= Y;
12         count <= 29'd0;
13         req_latch <= 1'b0;
14     end
15 Y: begin
16     ...
```

Listing 1: modified state transition

In this design, an intermediate 1-bit register `req_latch` is created. This acts as a temporary storage of REQUEST signals. No matter how many times the button is pressed, the latch keeps to be 1 until state transition happens.

State G is now kept to last at least 10 seconds before it transits to Y. When it passes more than

10 seconds and either a REQUEST button is pressed or there is a stored REQUEST signal, G transits to Y and `req_latch` is initialized to 0.

As a result, if the button is pressed for multiple times, the vehicle green light will be ON for at least 10 seconds before it turns yellow, avoiding the case that traffic is blocked due to multiple pedestrian press.

However, this implies that even though there is no vehicle on the lane, the pedestrian will still needs to wait for 10 seconds after pressing the button, if the pedestrian unfortunately arrives right after the pedestrian light turning red.

The expected wait time is definitely lower than 10 seconds, since the pedestrian could arrive anytime, and it is independent of the state transition of the traffic light controller. If the pedestrian arrives more than 10 seconds after the vehicle light turning green, no wait time is needed under this situation. The evaluation of expected wait time requires extra statistical modelling.

4 Conclusion

In this laboratory activity, a pedestrian light-controlled crossing system was successfully implemented and verified on the DE1-SoC FPGA platform. By applying Finite State Machine design principles, the required transition logic between vehicle and pedestrian lights was established and validated through hardware testing. The use of Quartus Prime for pin assignment and FPGA configuration ensured that the digital design correctly interfaced with physical peripherals, such as push-buttons for inputs and LEDs for status display.

Beyond the basic implementation, a significant limitation regarding traffic flow efficiency was identified in the original design. By introducing a `req_latch` mechanism and a minimum duration constraint for the vehicle Green state, the system was made more robust against frequent pedestrian requests. This improvement prevents potential traffic congestion while maintaining

system operation. Although this introduces a deterministic waiting period for pedestrians, it represents a necessary trade-off in real-world infrastructure design. Future enhancements could involve more sophisticated sensor integration or machine-learning based control of traffic light states.

5 Appendix

Original code provided by lab handout:

```
1  module tlc (  
2      input wire clk,  
3      input wire request,  
4      input wire reset,  
5      output reg [4:0] \output  
6  );  
7      localparam G = 2'd0;  
8      localparam Y = 2'd1;  
9      localparam R = 2'd2;  
10     reg [1:0] state;  
11     reg [28:0] count;  
12     always @(posedge clk or negedge reset) begin  
13         if (!reset) begin  
14             state <= G;  
15             count <= 29'd0;  
16         end else begin  
17             case (state)  
18                 G: begin  
19                     if (request == 1'b0) begin  
20                         state <= Y;
```



```

21         count <= 29'd0;
22     end
23 end
24 Y: begin
25     if (count == 29'd250000000) begin
26         state <= R;
27         count <= 29'd0;
28     end else begin
29         count <= count + 29'd1;
30     end
31 end
32 R: begin
33     if (count == 29'd500000000) begin
34         state <= G;
35         count <= 29'd0;
36     end else begin
37         count <= count + 29'd1;
38     end
39 end
40 default: begin
41     state <= G;
42     count <= 29'd0;
43 end
44 endcase
45 end
46 end
47 always @(*) begin
48     case (state)

```

```
49         G: \output = 5'b10001;
50         Y: \output = 5'b01001;
51         R: \output = 5'b00110;
52         default: \output = 5'b10001;
53     endcase
54 end
55 endmodule
```

Listing 2: original implementation of traffic light controller